



DOCUMENTACION BACKEND

“SISTEMA DE RESERVAS DE SALAS IPSS”

1. INTRODUCCION

El sistema de reservas de salas IPSS, describe el desarrollo de una API RESTfull. El proyecto fue implementado utilizando Node Js, Express y Sequelize ORM en un entorno local.

El objetivo del sistema es permitir la gestión de usuarios, salas y reservas mediante endpoints independientes, utilizando métodos HTTP como GET; POST; PUT; PATCH y DELETE. Además, se incorporó autenticación mediante token JWT, validación de roles, migraciones, seeders y reglas de negocio para validar horarios y disponibilidad de salas.

La API fue organizada bajo el patrón MVC, separando modelos, controladores, rutas y middlewares. Las pruebas fueron realizadas mediante Postman, verificando que las respuestas fueran entregadas en formato Json y que las operaciones se reflejaran correctamente en la base de datos.

2. OBJETIVOS DEL BACKEND

Es construir una API RESTfull funcional para administrar un sistema de reserva de salas, conectada a una base de datos MYSQL y organizada bajo el patron MVC.

Objetivo	Descripción
Crear una API RESTful	Desarrollar un backend con Node.js y Express que permita recibir y responder peticiones HTTP.
Conectar con base de datos	Utilizar MySQL como base de datos para almacenar usuarios, salas y reservas.
Aplicar patrón MVC	Separar el proyecto en modelos, controladores, rutas y middlewares para mantener una estructura ordenada.
Utilizar Sequelize ORM	Trabajar con la base de datos mediante modelos de JavaScript.
Crear migraciones	Generar las tablas del sistema desde el código, sin crearlas manualmente en phpMyAdmin.
Crear seeders	Insertar datos iniciales de prueba para usuarios, salas y reservas.
Implementar login	Permitir el inicio de sesión mediante correo y contraseña.
Generar token JWT	Entregar un token al usuario cuando inicia sesión correctamente.
Proteger rutas	Validar el token antes de permitir el acceso a ciertos endpoints.
Controlar roles	Diferenciar acciones entre usuarios con rol admin y usuario.
Gestionar usuarios	Crear, listar, actualizar, cambiar estado y eliminar lógicamente usuarios.
Gestionar salas	Crear, listar, actualizar, cambiar estado y eliminar lógicamente salas.

Gestionar reservas	Crear, listar, actualizar, cambiar estado y cancelar reservas.
Validar horarios	Verificar que las reservas estén dentro del horario permitido.
Validar disponibilidad	Evitar que una sala sea reservada si ya tiene una reserva en el mismo horario.
Responder en JSON	Entregar respuestas claras en formato JSON para cada operación.
Probar con Postman	Verificar el funcionamiento de los endpoints mediante pruebas en Postman.

3. TECNOLOGIAS UTILIZADAS

Se utilizaron distintas tecnológicas orientadas a la creación y son:

Tecnología / Herramienta	Función dentro del proyecto
Node.js	Se utilizó como entorno de ejecución para desarrollar el backend con JavaScript.
Express	Permitió crear el servidor, configurar rutas y manejar las peticiones HTTP.
MySQL	Fue utilizado como sistema de base de datos para almacenar usuarios, salas y reservas.
Sequelize ORM	Permitió trabajar con la base de datos mediante modelos de JavaScript.
Sequelize CLI	Se utilizó para crear y ejecutar migraciones y seeders.
Migraciones	Permitieron crear las tablas del sistema de forma ordenada desde el código.
Seeders	Se usaron para cargar datos iniciales de prueba en la base de datos.
JWT	Se utilizó para generar tokens de autenticación al iniciar sesión.
bcryptjs	Permitió encriptar las contraseñas antes de guardarlas en la base de datos.

dotenv	Se utilizó para manejar variables de entorno, como los datos de conexión y la clave del token.
cors	Permitió habilitar solicitudes desde otros orígenes, pensando en la conexión con el frontend.
nodemon	Se utilizó durante el desarrollo para reiniciar automáticamente el servidor al realizar cambios.
Postman	Se utilizó para probar los endpoints, enviar datos en formato JSON y revisar las respuestas del backend.

4. ESTRUCTURAS DEL BACKEND

La estructura del backend es la siguiente:

Carpeta / Archivo	Tipo	Función principal
reservas_salas2_mvc/	Carpeta raíz	Contiene todo el backend del sistema.
config/	Carpeta	Guarda la configuración de Sequelize.
config/config.js	Archivo JS	Permite conectar Sequelize con la base de datos usando variables del .env.
migrations/	Carpeta	Guarda las migraciones que crean las tablas en MySQL.
seeders/	Carpeta	Guarda datos de prueba para usuarios, salas y reservas.
src/	Carpeta	Contiene el código principal del backend.
src/app.js	Archivo JS	Configura Express, middlewares y rutas principales.
src/controllers/	Carpeta	Contiene la lógica de cada módulo.
authController.js	Archivo JS	Controla el login y genera el token JWT.
usuarioController.js	Archivo JS	Gestiona el CRUD de usuarios.
salaController.js	Archivo JS	Gestiona el CRUD de salas.
reservaController.js	Archivo JS	Gestiona reservas, horarios y disponibilidad.

src/models/	Carpeta	Contiene los modelos de Sequelize.
usuario.js	Archivo JS	Modelo de la tabla usuarios.
sala.js	Archivo JS	Modelo de la tabla salas.
reserva.js	Archivo JS	Modelo de la tabla reservas.
src/routes/	Carpeta	Contiene las rutas de la API.
authRoutes.js	Archivo JS	Define rutas de autenticación.
usuarioRoutes.js	Archivo JS	Define rutas de usuarios.
salaRoutes.js	Archivo JS	Define rutas de salas.
reservaRoutes.js	Archivo JS	Define rutas de reservas.
src/middlewares/	Carpeta	Contiene validaciones intermedias.
authMiddleware.js	Archivo JS	Valida el token JWT.
roleMiddleware.js	Archivo JS	Valida permisos por rol.
server.js	Archivo JS	Inicia el servidor y prueba conexión con MySQL.
.env	Archivo config	Guarda variables de entorno del proyecto.
.env.example	Archivo config	Sirve como ejemplo de configuración.
README.md	Archivo MD	Documenta cómo usar el proyecto.

Esta estructura ayuda a mantener el proyecto más ordenado, ya que cada archivo tiene una responsabilidad específica. Además, permite que el backend sea más fácil de revisar, probar y mantener durante el desarrollo.

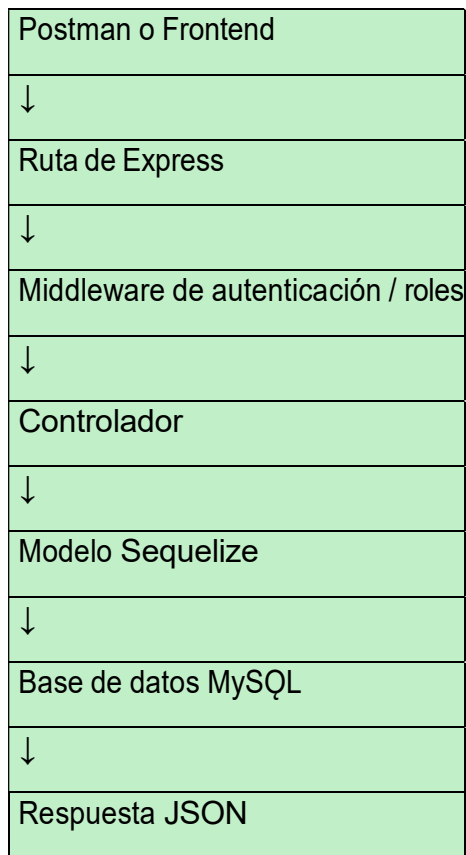
5. FLUXO DE CONEXIÓN DE LA API

El backend funciona recibiendo peticiones HTTP desde Postman o desde un futuro frontend. Estas peticiones llegan primero a las rutas definidas en el proyecto, luego pasan por los middlewares si corresponde, después llegan al controlador y finalmente interactúan con los modelos Sequelize para consultar o modificar la base de datos.

Etapa	Descripción
Cliente / Postman	Envía una petición HTTP al backend, por ejemplo GET, POST, PUT, PATCH o DELETE.
Ruta	Recibe la petición y determina qué controlador debe ejecutarse.
Middleware	Valida información antes de llegar al controlador, como el token JWT o el rol del usuario.
Controlador	Contiene la lógica principal de la acción solicitada. Valida datos, llama al modelo y prepara la respuesta.
Modelo Sequelize	Representa las tablas de la base de datos y permite consultar, crear, actualizar o cambiar registros.
Base de datos MySQL	Guarda la información de usuarios, salas y reservas.
Respuesta JSON	El backend devuelve una respuesta clara en formato JSON indicando si la operación fue exitosa o si ocurrió un error.

Por ejemplo, cuando se crea una reserva, Postman envía los datos al endpoints correspondiente. Luego el backend valida que el usuario exista, que la sala este disponible, que el horario sea valido y que no exista otra reserva en el mismo horario. Si todo esta correcto. La reserva se almacena en la base de datos y el sistema devuelve una respuesta Json indicando que la reserva fue creada correctamente.

Este flujo permite que el frontend no acceda directamente a la base de datos, sino que todas las operaciones pasen por backend, manteniendo mayor orden, control y seguridad.



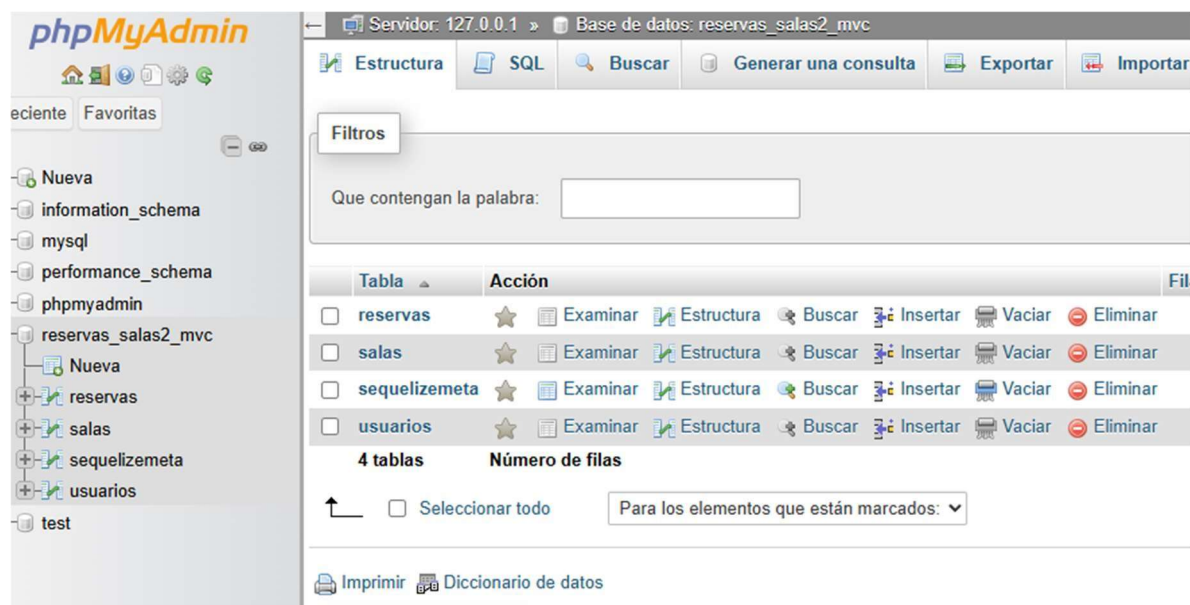
6. BASE DE DATOS UTILIZADA

Para el desarrollo del backend se utilizó una base de datos MySQL, creada en un entorno local mediante XAMPP y administrada desde phpMySQL.

El nombre de la base de datos es: **reservas_salas2_mvc**

En esta base de datos se almacena la información principal del sistema, permitiendo guardar usuarios, salas y reservas. A diferencia de una creación Manual de tablas, es este proyecto las tablas fueron generadas mediante migraciones de Sequelize lo que permite mantener un mayor control sobre la estructura de la base de datos.

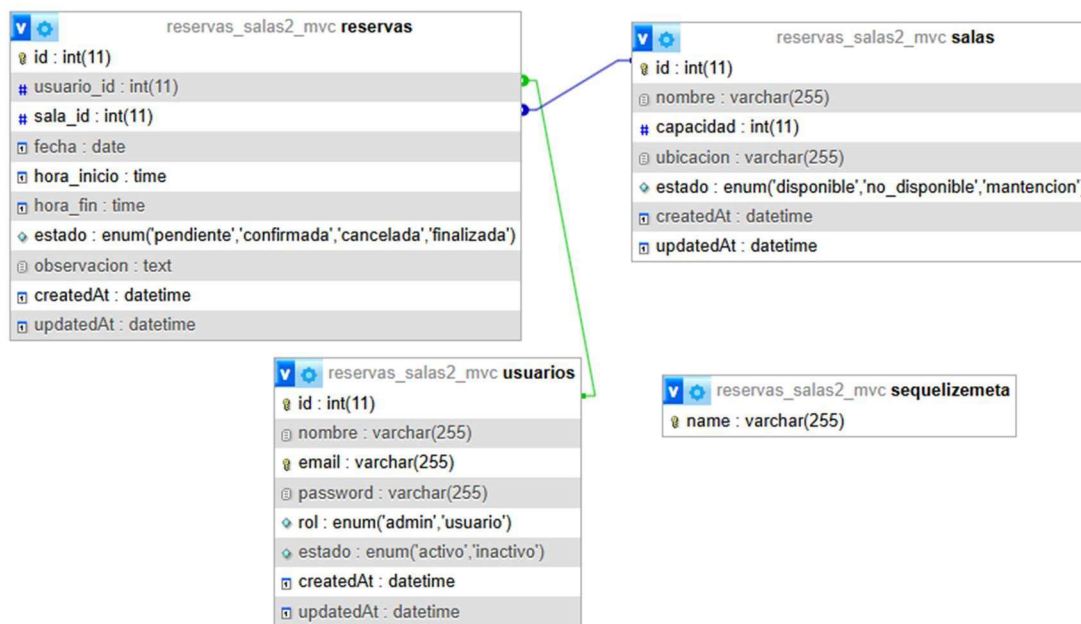
Las tablas utilizadas son



RELACIONES ENTRE TABLAS

En el sistema se configuraron relaciones mediante Sequelize:

Estas relaciones permiten que al listar reservas también se pueda obtener información del usuario y de la sala asociada, lo que hace que la respuesta de la API sea más completa y útil.



Relación	Descripción
Usuario tiene muchas reservas	Un usuario puede realizar varias reservas.
Reserva pertenece a un usuario	Cada reserva está asociada a un usuario.
Sala tiene muchas reservas	Una sala puede tener varias reservas en distintas fechas u horarios.
Reserva pertenece a una sala	Cada reserva está asociada a una sala específica.

7. MIGRACIONES Y SEEDERS

Se utilizaron migraciones y seeders mediante Sequelize CLI. Esto permitió trabajar la base de datos de forma más ordenada, evitando crear las tablas manualmente desde phpMyAdmin.

MIGRACIONES: Se utilizaron para crear la estructura de las tablas principales del sistema. En este caso se crearon migraciones para las tablas

Migración	Función
create-usuarios	Crea la tabla usuarios.
create-salas	Crea la tabla salas.
create-reservas	Crea la tabla reservas y sus relaciones con usuarios y salas.

Para ejecutar las migraciones se utilizó el comando:

`npx.cmd sequelize-cli db:migrate`

Al ejecutar este comando Sequelize creó las tablas en la base de datos reservas_sala2_mvc

También se creó automáticamente la tabla SequelizeMeta, que es utilizada por Sequelize para registrar que migraciones ya fueron ejecutadas.

Seeders:

Los seeders se utilizaron para cargar datos iniciales de prueba en la base de datos. Esto permitió probar el sistema sin tener que ingresar a todos los datos manualmente.

Seeder	Datos cargados
demo-usuarios	Carga un usuario administrador y un usuario normal.
demo-salas	Carga salas de prueba.
demo-reservas	Carga reservas iniciales asociadas a un usuario y una sala.

Para ejecutar los seeders se utilizó el comando:

```
npx.cmd sequelize-cli db:seed:all
```

Los usuarios de pruebas son:

Rol	Correo	Contraseña
admin	admin@reservas.cl	admin123
usuario	usuario@reservas.cl	usuario123

El uso de seeders ayudó a probar el login los roles, las reservas y la disponibilidad de salas desde Postman.

IMPORTANCIA DE USAR MIGRACIONES:

El uso de migraciones permitió controlar la creación de las tablas desde el código. Por otro lado, los seeders permitieron cargar datos de prueba de forma rápida y repetible. Esto fue útil durante el desarrollo, ya que permitió crear la base de datos, probar el sistema y mantener una estructura más ordenada.

8. TRANSACCIONES, COMMIT Y ROLBACK

En el controlador de reservas se agregó el uso de transacciones con Sequelize para proteger algunas acciones importantes del sistema, como crear, actualizar, cambiar estado y cancelar una reserva.

Esto significa que, si una operación se realiza correctamente, los cambios se guardan en la base de datos mediante commit. En cambio, si ocurre un error durante el proceso, se usa rollback para deshacer la operación y evitar que queden datos incompletos.

Esta modificación ayuda a que el módulo de reservas sea más seguro, ya que antes de guardar una reserva se revisa que el usuario exista, que la sala esté disponible, que el horario sea válido y que no exista otra reserva en el mismo horario.

EVIDENCIA DE VALIDACION POSTERIOR A LAS MODIFICACIONES

Después de realizar ajustes en el controlador de reservas, se volvieron a probar las operaciones principales del módulo desde Postman. Estas pruebas permitieron verificar que el sistema continuara funcionando correctamente después de incorporar transacciones con Sequelize.

Las capturas de pantalla de estas pruebas se encuentran en el Anexo de Desarrollo y Pantallazos, donde se muestra la ejecución de los endpoints y sus respuestas correspondientes.

9. ENDOPOINTS IMPLEMENTADOS

Los endpoints fueron organizados en módulos para tener una mejor estructura y fueron probados utilizando Postman. Para cada prueba se verificó el método HTTP, la URL, el envío de datos en formato JSON cuando correspondía y la respuesta entregada por el sistema.

Las pruebas permitieron comprobar que la API respondiera correctamente, que los datos se guardaran en MySQL y que las validaciones funcionaran según las reglas definidas.

- **Módulo Auth**

Método	Endpoint	Descripción
POST	http://localhost:3000/api/auth/login	Permite iniciar sesión y obtener un token JWT.
GET	http://localhost:3000/api/auth/perfil	Permite validar si el token enviado es correcto.

Prueba de login correcto

Se probó el inicio de sesión utilizando el correo y contraseña del usuario administrador cargado por el seeders

Body Utilizado:

```
{  
  "email":  
    "admin@rese  
    rvas.cl",  
  "password":  
    "admin123"  
}
```

El sistema respondió correctamente entregando un token JWT y los datos básicos del usuario. Esto permitió comprobar que el login funciona y que las contraseñas encriptadas pueden ser validadas correctamente.

PRUEBA DE LOGIN INCORRECTO

Se realizó una prueba enviando una contraseña incorrecta. El sistema respondió con un mensaje de error indicando que las credenciales eran incorrectas. Esto permitió comprobar el manejo de errores en el inicio de sesión.

- **Modulo Usuarios**

Método	Endpoint	Descripción
GET	http://localhost:3000/api/usuarios	Lista los usuarios registrados.
GET	http://localhost:3000/api/usuarios/:id	Busca un usuario por su ID.
POST	http://localhost:3000/api/usuarios	Crea un nuevo usuario.
PUT	http://localhost:3000/api/usuarios/:id	Actualiza los datos de un usuario.
PATCH	http://localhost:3000/api/usuarios/:id/estado	Cambia el estado de un usuario.
DELETE	http://localhost:3000/api/usuarios/:id	Realiza eliminación lógica del usuario.

PRUEBAS REALIZADAS EN USUARIOS:

Se probó el listado de usuarios, la creación de un nuevo usuario, la actualización de sus datos, el cambio de estado y la eliminación lógica.

También se validó que el sistema no permitiera crear un usuario con un correo ya registrado. En ese caso, la API respondió con un error indicando que el correo ya existía.

- **Modulos Salas**

Método	Endpoint	Descripción
GET	http://localhost:3000/api/salas	Lista las salas registradas.
GET	http://localhost:3000/api/salas/:id	Busca una sala por su ID.
POST	http://localhost:3000/api/salas	Crea una nueva sala.
PUT	http://localhost:3000/api/salas/:id	Actualiza los datos de una sala.
PATCH	http://localhost:3000/api/salas/:id/estado	Cambia el estado de una sala.
DELETE	http://localhost:3000/api/salas/:id	Realiza eliminación lógica de una sala.

PRUEBAS REALIZADAS EN SALAS

Se probó el listado de salas cargadas por el seeders y luego se creó una nueva sala desde Postman.

También se realizaron pruebas para actualizar la sala, cambiar sus estado a mantención y eliminarla lógicamente.

En este caso, la eliminación no borra físicamente el registro, sino que cambia el estado de la sala a no disponible.

Estas pruebas permitieron comprobar que el módulo de salas funciona correctamente y que los cambios se reflejan en la base de datos.

- **Módulos reservas**

Método	Endpoint	Descripción
GET	http://localhost:3000/api/reservas	Lista las reservas registradas.
GET	http://localhost:3000/api/reservas/:id	Busca una reserva por su ID.
POST	http://localhost:3000/api/reservas	Crea una nueva reserva.
PUT	http://localhost:3000/api/reservas/:id	Actualiza una reserva.
PATCH	http://localhost:3000/api/reservas/:id/estado	Cambia el estado de una reserva.
DELETE	http://localhost:3000/api/reservas/:id	Cancela una reserva de forma lógica.
GET	http://localhost:3000/api/reservas/disponibilidad	Consulta si una sala está disponible en una fecha y horario.

PRUEBAS REALIZADAS EN RESERVAS:

Se probó el listado de reservas, donde el sistema mostro la información de la reserva junto con los datos del usuario y la sala relacionada.

Luego se creó una nueva reserva indicando usuario, sala, fecha, hora de inicio, hora de término y observación. El sistema valido que el usuario existiera, que la sala estuviera disponible y que el horario fuera correcto.

También se realizaron pruebas para actualizar una reserva, cambiar su estado y cancelarla mediante eliminación lógica.

PRUEBA DEL HORARIO OCUPADO:

Se intentó crear una reserva en una sala que ya tenia una reserva en un horario solapado. El sistema respondió con un error indicando que la sala no estaba disponible en ese horario.

Esta prueba permitió comprobar que el backend evita reservas duplicadas o cruzadas

PRUEBA DEL HORARIO PERMITIDO:

Se intento crear una reserva fuera del horario permitido, por ejemplo entre las 19:00hrs y 20:00hrs. El sistema respondió con un mensaje indicando que las reservas solo pueden realizarse entre las 08:00 y las 18:00hrs. Permitiendo comprobar la validación de horarios definida en el controlador de reservas.

PRUEBA DE DISPONIBILIDAD

Ejemplo:

http://localhost:3000/api/reservas/disponibilidad?sala_id=1Cfecha=2026-05-05Chora_inicio=09:30Chora_fin=10:30

El sistema respondió indicando si la sala estaba disponible o no en ese horario. Esta prueba fue importante porque demuestra que el backend puede consultar la disponibilidad antes de crear una reserva.

PRUEBA RESERVA OCUPADA

Metodo GET

http://localhost3000/api/reservas/disponibilidad?sala_id=1&fecha=2025-05-05&hora_inicio=09:30&hora_fin=10:30

- Se crearon los archivos:

.env.example

Readme

10. TABLA CODIGOS HTTP UTILIZADOS

Código HTTP	Nombre	Uso dentro del sistema
200	OK	Se utiliza cuando una solicitud se ejecuta correctamente, por ejemplo al listar usuarios, salas o reservas, actualizar datos, cambiar estados o cancelar una reserva.
201	Created	Se utiliza cuando se crea un nuevo registro correctamente, como un usuario, una sala o una reserva.
400	Bad Request	Se utiliza cuando los datos enviados son inválidos o incompletos, por ejemplo campos obligatorios vacíos, horario fuera del rango permitido o estado no válido.
401	Unauthorized	Se utiliza cuando el usuario no está autorizado, por ejemplo cuando no se envía token JWT, el token es inválido o está expirado.
404	Not Found	Se utiliza cuando no se encuentra un recurso solicitado, como un usuario, sala o reserva inexistente.
409	Conflict	Se utiliza cuando existe un conflicto con los datos, por ejemplo al intentar registrar un correo ya existente o crear una reserva en un horario ocupado.
500	Internal Server Error	Se utiliza cuando ocurre un error interno del servidor o un problema inesperado durante la ejecución de la solicitud.

11. DECLARACIÓN DE USO DE IA

En este proyecto se utilizó inteligencia artificial como apoyo para resolver dudas, ordenar ideas y mejorar la redacción de la documentación.

Su uso fue complementario al trabajo realizado durante el desarrollo, pruebas y revisión del sistema.